

MKT4822 — Reinforcement Learning Term Project

****Project topic:**** Developing an RL-Based Decision Mechanism for a Local LLM Coding Agent

****Source repository:**** `mkt4822-RL-term-project-agent-cli.zip`

****Final deadline:**** ****June 8, 2026****

****Note:**** The final deadline is strict and will not change.

****Submission channels:**** Classroom + `online.yildiz.edu.tr`

****Report format:**** `REPORT.pdf`

****Execution environment:**** WSL

1. General Project Objective

In this project, you must improve only the ****reinforcement-learning-related decision mechanism**** of the provided local LLM coding-agent repository. The system generates `algorithm.py` from a natural-language task and then runs automatic validation and runtime checks.

The objective of this assignment is not to modify the LLM model, validation system, agent loop, or benchmark infrastructure. Your task is only to work on the permitted RL files and improve how the agent chooses generation strategies in different contexts.

For any case not explicitly defined in this document, interpretation, implementation, and final evaluation authority belongs to the instructor. Submission validity, scope violations, originality, group contribution, technical error handling, and grading interpretation are determined by the instructor.

2. Base Code, License, and Reference Requirement

The base repository is provided by the instructor for this course. The `LICENSE` and `NOTICE.md` files define this as limited, course-internal educational use. Public

availability of the code does not mean it can be freely redistributed, commercially used, or presented as a fully original from-scratch project.

All repository materials must be used only for this course project.

For clarity, "all repository materials" includes the entire repository content: code, config files, task files, scripts, documents, and related project artifacts.

You must explicitly reference the base repository in your report. A statement similar to the following must appear in `REPORT.pdf`:

```text

This work was developed on top of the mecha-agent-cli codebase provided by the instructor

within the MKT4822 course. Only the permitted reinforcement-learning files were modified

for this project. Agent loop, validation, LLM backend, CLI, benchmark, config, and task infrastructure were preserved.

```

The `LICENSE` file cannot be removed, edited, or bypassed. Using the code outside course scope, redistributing it, or migrating it into another project is not considered permitted.

3. Group Rules

This is a group project.

```text

Group size must be at least 8 and at most 10 students.

...

Each group must select one **group leader**. The leader is responsible for verifying the final submission package, submitting on Classroom, ensuring contribution details are included in the report, and representing the group in communication with the instructor.

`REPORT.pdf` must include the following:

```text

Group name or group number

Group leader: full name and student ID

All group members: full names and student IDs

A short note explaining how the group leader was selected

Contribution summary for each group member

...

4. Submission Channels

The project must be submitted through both Classroom and `online.yildiz.edu.tr`.

Submission format:

```text

Each student uploads the same submission package individually to online.yildiz.edu.tr.

Only the group leader uploads the same package as the group submission on Classroom.

If group members upload different package contents, the project grade is 0.

...

This ensures submission through both channels. Students other than the group leader are not expected to submit separately to Classroom, but every student must submit individually to `online.yildiz.edu.tr` .

Deadline rule: the final deadline is the same for Classroom and `online.yildiz.edu.tr` , strict, and non-changeable.

## ## 5. Environment and Portability

Evaluation will be performed in WSL. Therefore, your solution must be Linux/WSL compatible and portable.

The following are prohibited:

```text

Windows absolute paths such as C:\Users\...

Drive paths such as D:\...

Personal Windows mount paths such as /mnt/c/Users/...

User-specific absolute paths such as /home/<personal_user>/...

Desktop path usage

Hard-coded workspace path

Hard-coded output path

```

Expected approach:

```text

Use relative paths

Generate files inside the working directory

Use platform-independent path logic

Do not depend on a specific username or computer

```\n

`generation\_report.csv` generated by `algorithm.py` must also be written to the current working directory via relative path. Personal or absolute paths will cause point deductions.

## ## 6. Permitted Source Files

Only the following production source files may be modified:

```text\n

src/mecha_agent_cli/learning/bandit.py

src/mecha_agent_cli/learning/context.py

src/mecha_agent_cli/learning/arm_registry.py

src/mecha_agent_cli/learning/reward.py

src/mecha_agent_cli/learning/__init__.py

src/mecha_agent_cli/learning/q_learning.py

```\n

No additional Python files, packages, modules, or production source code may be created outside this scope.

`q\_learning.py` may only support the agent's RL decision mechanism. You may not place direct `algorithm.py` solutions, hard-task solutions, or ready-to-inject prompt source code in this file.

Additional scope rule:

```text

Students may improve only the provided learning files; replacing them with a completely new from-scratch learning system is not allowed.

```

## ## 7. Prohibited Directories and Files

You must not modify the following directories:

```text

src/mecha_agent_cli/agent/

src/mecha_agent_cli/app/

src/mecha_agent_cli/config/

src/mecha_agent_cli/core/

src/mecha_agent_cli/llm/

src/mecha_agent_cli/memory/

src/mecha_agent_cli/repo/

src/mecha_agent_cli/sandbox/

src/mecha_agent_cli/ui/

src/mecha_agent_cli/validation/

configs/

project-assignment/

scripts/

docs/

tests/

```

The following files are also protected and cannot be modified:

```text

README.md

LICENSE

NOTICE.md

AGENTS.md

pyproject.toml

pyrightconfig.json

.env.example

uv.lock

```

Explicitly prohibited actions:

```text

Modifying agent loop code

Modifying validation code

Modifying LLM backend code

Modifying benchmark scripts

Modifying task files

Modifying config files to gain advantage

Changing the model name

Disabling runtime or validation checks

Manually editing algorithm.py

Injecting ready-made code via prompts to generate algorithm.py

Writing hard-coded task solutions

Using external APIs or internet services (except `ollama pull qwen3:4b` and Python package installation)

Creating new source code files outside q_learning.py

```

Exception (setup/network use):

```text

Only `ollama pull qwen3:4b` is allowed for model download.

Installing Python libraries for project setup is allowed.

No other external API or internet service is allowed during project runs and evaluation.

```

## ## 8. Ready-Made Code in Prompts Is Forbidden

One of the most important rules is: students may not manually write `algorithm.py`, and may not paste ready working code into prompts to have the LLM regenerate it.

Forbidden examples:

```text

Pasting a full working solution into the prompt

Using code taken from the internet or another group

Providing complete class implementations

Providing complete function bodies

Providing a nearly complete scaffold and filling only blanks

Using long pseudocode that is effectively direct code

Embedding algorithm.py solution content in prompts

Forcing hard-coded solution generation

```



Allowed prompt approach:

```text

Natural-language problem description

High-level modeling objective

Expected output types

General method naming

High-level requirements such as train/test/evaluation/visualization/report

Natural-language file-generation and execution constraints

```

Your report must include a prompt usage statement. The instructor may request prompt history, task text, or additional evidence of the generation process.

## ## 9. Error Analysis Responsibility

Syntax errors, import errors, runtime errors, validation failures, behavior errors, benchmark errors, WSL path errors, and model generation errors must be analyzed by students. The instructor will not provide ready-made patches, ready-made algorithms, or ready-made RL solutions.

Students must report the success of their RL-side improvements together with error analysis. Evaluation is based not only on whether the code runs, but also on observed failures and how they are handled.

## ## 10. Setup and Baseline Checks

Recommended initial setup:

```
```bash
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -e ".[dev]"
```
```

Ollama and the `qwen3:4b` model are expected to be available for local LLM runs:

```
```bash
ollama pull qwen3:4b
```
```

Baseline checks:

```
```bash
python -m compileall -q src scripts
ruff check .
ruff format --check .
pyright
pytest -q
```
```

Config check:

```
```bash
PYTHONPATH=src python -m mecha_agent_cli config show --path .
```
```

``.venv/`, `.mecha-agent/`, __pycache__/, `.pytest_cache/`, `.ruff_cache/`, `.mypy_cache/`, large binary files, and personal workspace folders must not be included in the submission package.`

## ## 11. Proof Runs in a Temporary Workspace

Students must not perform proof runs by manually adding `algorithm.py` into the repository root. Proof runs must be performed in a temporary workspace (for example under `runs/`).`

For testing, only the learning method may be changed in config, and learning may be enabled.

This means in temporary workspace config, `learning.mode`` may be changed and `learning.enabled: true`` may be set. Other config behavior must remain aligned with project defaults. This does not mean changing repository main config files; it applies only to temporary evaluation workspaces.

Example workspace config logic:

```
```yaml
```

```
repo:
```

```
  target_file: algorithm.py
```

```
agent:
```

```
  max_repairs: 3
```

```
memory:
```

```
  path: .mecha-agent/memory.sqlite
```

direct:

request_timeout_sec: 600

learning:

enabled: true

mode: "thompson"

persist: true

```

## ## 12. Experimental Comparison

Each group must compare at least the following three systems:

```text

A0: baseline with learning disabled

A1: instructor-provided starter repository with the task text from `project-assignment/mkt4822\_actor\_critic.txt`

A2: the group's improved RL structure

```

Minimum required metrics:

```text

success_rate

runtime_pass_rate

mean_attempts

mean_reward

mean_duration_sec

Distribution of syntax_fail / import_fail / runtime_fail / behavior_fail / other_fail

```

Metric definitions (mandatory):

```text

success_rate = successful validations / total run count

runtime_pass_rate = runtime-pass count / total run count

mean_attempts = average attempts per run

mean_reward = average final reward per run

mean_duration_sec = average wall-clock seconds per run

failure distribution = counts of syntax_fail, import_fail, runtime_fail, behavior_fail,
other_fail

```

Fairness protocol (mandatory):

```text

All official grading runs (A0, A1, A2, and final check) are executed on the instructor's own computer.

The same machine and environment settings must be used across A0/A1/A2.

Student local results are informative only and are not solely authoritative for grading.

```

General benchmark command:

```bash

```
python scripts/hard_runtime_benchmark.py --backend ollama --mode both --max-
attempts 6 --repeat 3 --out runs/student_benchmark.jsonl --workspaces-dir
runs/student_workspaces
```

```
```
```

The fake backend is for smoke testing only and is not sufficient as final performance evidence:

```
```bash
```

```
python scripts/hard_runtime_benchmark.py --backend fake --mode both --max-
attempts 2 --repeat 1 --out runs/fake_smoke.jsonl --workspaces-dir
runs/fake_smoke_workspaces
```

```
```
```

### ## 13. Submission Package Structure

Your submission package must follow this structure:

```
```text
```

```
submission/
```

```
├─ src/
|   └─ mecha_agent_cli/
|       └─ learning/
|           ├─ bandit.py
|           ├─ context.py
|           ├─ arm_registry.py
|           ├─ reward.py
|           ├─ __init__.py
|           └─ q_learning.py
```

```

└─ REPORT.pdf
└─ runs/
|   └─ original_or_baseline_results.jsonl
|   └─ original_or_baseline_results.summary.json
|   └─ student_results.jsonl
|   └─ student_results.summary.json
└─ proof/
    └─ generated_algorithm.py
    └─ run_output.txt
    └─ validate_output.txt
    └─ generation_report.csv
    └─ prompt_or_task_summary.txt
...

```

`generated\_algorithm.py` must be a copy of `algorithm.py` generated by the agent in a temporary workspace. Manually corrected `algorithm.py` submissions are not allowed.

14. REPORT.pdf Content

The report must be submitted only in PDF format. `REPORT.pdf` must include the following sections:

1. Cover page: course name, project name, group name, group leader, members, student IDs, submission date, base repo reference.
2. Source and scope statement: base repository provided by instructor, only permitted RL files modified, no ready-made prompt code used.
3. Group organization: leader selection, task distribution, member contributions.
4. Modified files table: only the six permitted files.

5. Implementation and academic/technical report: purpose of each file, addressed problem, RL relation, bandit/q_learning relation, test method, references.
6. Prompt usage statement: no ready-made code, natural-language task steering.
7. Experimental protocol: OS, WSL details, Python version, Ollama model, hardware, benchmark command, max_attempts, repeat.
8. Results tables: A0/A1/A2 comparison, failure distribution, runtime and success metrics.
9. Hard task proof result: task file, attempt count, run/validate summary, generated algorithm location, CSV output.
10. Limitations and discussion: where improvements occurred, what remained unresolved, and what was learned.
11. References: textbook, papers, lecture notes, official documentation, base repository.

15. Grading

Total: 100 points

15.1 Instructor Run Attempt — 50 points

This part is evaluated through an instructor-run attempt in the instructor's WSL environment. A successful result on the student's own computer is not sufficient by itself.

To ensure fairness, all official evaluations are executed on the instructor's own computer.

15.2 Scope and Rule Compliance — 10 points

```text

Only permitted source files are modified.



Protected folders are untouched.

No new source files are created outside q\_learning.py.

No ready-made code is inserted in prompts.

LICENSE and NOTICE are preserved.

```

15.3 Experimental Comparison — 15 points

```text

A0/A1/A2 comparison

Benchmark commands

Success and runtime metrics

Error analysis

Result interpretation

```

15.4 PDF Report and Implementation Report — 15 points

```text

REPORT.pdf must be complete.

Each permitted file must be explained separately.

Bandit and q\_learning.py modeling process must be documented.

Academic/technical references must be provided.

Prompt usage statement must be included.

```

15.5 Submission Process — 10 points

```text

Each student must upload individually to [online.yildiz.edu.tr](https://online.yildiz.edu.tr).

Group leader must submit the group package on Classroom.

Submission structure must match the required format.

If group members upload different package contents, the project grade is 0.

No unnecessary large/temporary files should be included.

```

16. Major Point Loss or Invalid Submission Reasons

The following are considered major scope violations:

```text

Modifying validation code

Modifying agent loop code

Modifying benchmark scripts

Modifying task files

Modifying LLM backend code

Modifying config files for unfair advantage

Manually editing algorithm.py

Regenerating from ready-made prompt code

Adding hard-coded task solutions

Modifying production code outside permitted files

Creating new source files outside q\_learning.py

Failing to reference the base repository

Modifying LICENSE or NOTICE

Using external APIs or internet services (except `ollama pull qwen3:4b`` and Python package installation)

Using paths incompatible with WSL evaluation

Copying code from other groups

Reporting fabricated experimental results

...

Non-original solutions, highly similar cross-group outputs, or hard-coded evaluation solutions will cause severe deductions. If necessary, cases may also be reviewed under academic ethics rules.

## ## 17. MKT4822 Hard Task Expectation

For this course, the hard task file in the starter repository is:

```text

project-assignment/mkt4822_actor_critic.txt

```

This task asks the local LLM to produce a **working Actor-Critic-based reinforcement learning implementation**. The generated `algorithm.py` is expected to include the following high-level components:

```text

Actor-Critic approach

Problem or environment modeling structure

Training pipeline

Test or evaluation pipeline

Visualization or textual learning summary

generation_report.csv output

Executable main execution body

```

`q\_learning.py` is a permitted file for improving the agent's RL decision mechanism only; it must not be used as the direct hard-task solution.

### ## 18. MKT4822 Proof Run Example

The following example is provided for local proof runs. Do not modify repository main config files; enable learning in a temporary workspace.

```bash

```
TASK="$(cat project-assignment/mkt4822_actor_critic.txt)"
```

```
WORKSPACE="runs/mkt4822_proof_workspace"
```

```
mecha-agent init "$WORKSPACE" --force
```

```
cat > "$WORKSPACE/configs/default.yaml" <<'YAML'
```

```
repo:
```

```
  target_file: algorithm.py
```

```
agent:
```

```
  max_repairs: 3
```

```
memory:
```

```
  path: .mecha-agent/memory.sqlite
```

```
direct:
```

```
  request_timeout_sec: 600
```

learning:

enabled: true

mode: "thompson"

persist: true

YAML

```
mecha-agent run "$TASK" --path "$WORKSPACE" --max-attempts 6 --judge-final | tee
"$WORKSPACE/run_output.txt"
```

```
mecha-agent validate --path "$WORKSPACE" --task "$TASK" | tee
"$WORKSPACE/validate_output.txt"
```

```

Required proof outputs:

```text

runs/mkt4822_proof_workspace/algorithm.py

runs/mkt4822_proof_workspace/generation_report.csv

runs/mkt4822_proof_workspace/run_output.txt

runs/mkt4822_proof_workspace/validate_output.txt

```

For submission, you may copy `algorithm.py` as `proof/generated\_algorithm.py`. This file must not be manually edited.

## 19. MKT4822 Instructor Run Attempt — 50-Point Breakdown

| Criterion | Points |

|---|---:|

`mecha-agent run` executes without errors in WSL	8
Agent generates `algorithm.py` for the hard task	7
Generated code contains a working Actor-Critic section	10
Train/test or evaluation flow works	8
Visualization or textual learning summary is produced	4
`generation\_report.csv` is produced with relative path	4
`mecha-agent validate` succeeds	5
`algorithm.py` is not manually edited and no ready-made prompt code is used	4

Instructor-side WSL result is authoritative.

## ## 20. MKT4822 Checklist

```text

- [] Group size is between 8 and 10.
- [] Group leader is selected.
- [] Each student will upload to online.yildiz.edu.tr.
- [] Group leader will submit on Classroom.
- [] Group members uploaded the same submission package (otherwise grade = 0).
- [] Only the six permitted RL files are modified.
- [] Hard task solution is not written inside q_learning.py.
- [] Ready-made code is not placed in prompts.
- [] project-assignment/mkt4822_actor_critic.txt is not modified.
- [] Generated algorithm.py is not manually edited.
- [] generation_report.csv is produced.
- [] For testing config, only learning.mode is changed and learning.enabled may be enabled.
- [] REPORT.pdf is prepared.

☐ A0/A1/A2 experiments are reported.

☐ Academic/technical references are included.

☐ LICENSE and NOTICE are preserved.

☐ .venv and cache files are excluded from submission package.